



INTERNATIONAL INSTITUTE OF
INFORMATION TECHNOLOGY

H Y D E R A B A D

17/02/2025

Reverse Engineering and Refactoring of Rudra's Subscription Service

By Team - 17

Sanket Adlak
2024204005

Yash Sonkar
2024801001

Priyank Nagarnaik
2024204011

Nitheesh Chandra
2024801002

Aditya Singh Rathore
2024204012

Faculty: Karthik Vaidhyanathan
Semester: Spring 2025
Course: CS6.401 Software Engineering

Contents

LIST OF FIGURES	II
LIST OF TABLES	III
1. INTRODUCTION	1
2. PROJECT OBJECTIVES	1
3. SYSTEM OVERVIEW	1
4. TASK 1: REVERSE ENGINEERING	2
4.1. Identification of Relevant Classes	2
4.1.1. Subscription and Content Subsystem	2
4.1.2. Feed Organization Subsystem	2
4.1.3. User Management Subsystem	3
4.2. UML Class Diagram	5
4.2.1. Subscription and Content Subsystem	5
4.2.2. Feed Organization Subsystem	7
4.2.3. User Management Subsystem	7
4.3. Observation and Comments	7
5. TASK 2: DESIGN SMELLS AND CODE METRICS	10
5.1. Design Smell Analysis	10
5.2. Code Metrics	11
5.3. Implications Discussions	14
6. TASK 3: REFACTORING	15
6.1. Addressing Design Smells	15
6.2. Code Metrics Post Refactoring	16
6.3. Analysis of Code Metrics after Refactoring	17
6.4. Leveraging LLMs for Refactoring	18
6.5. Automation of Refactoring Pipeline	20
7. ACKNOWLEDGEMENTS	21
7.1. Contributions	21
APPENDICES	22
Appendix B: Additional UML Diagrams	23

List of Figures

Figure 1: Class Diagram of Subscription and Content Subsystem	6
Figure 2: Class Diagram of Feed Organization Subsystem	7
Figure 3: Class Diagram of Indexing Service	7
Figure 4: Class Diagram of User Management Subsystem	7
Figure 5: C3 before refactoring	11
Figure 6: Metrics before refactoring	12
Figure 7: SonarQube Stats before refactoring	12
Figure 8: SonarQube Issues before refactoring	13
Figure 9: SonarQube Issues before refactoring	13
Figure 10: SonarQube Issues before refactoring	14
Figure 11: Design smells given by Designite	16
Figure 12: C3 after refactoring	16
Figure 13: Metrics after refactoring	17
Figure 14: UML before refactoring.	23

List of Tables

Table 1: Design Smells Identified in the RSS Reader	10
Table 2: Comparison b/w our vs LLM refactoring	20

1. Introduction

The goal of this project is to analyze, document, and improve the existing Rudra's Subscription Service (RSS) Reader. The project involves reverse engineering the current codebase, identifying design smells and code metrics, and implementing refactoring strategies. The project aims to improve the maintainability, readability, and performance of the RSS Reader while applying software engineering principles learned in the course.

2. Project Objectives

- Analyze the existing RSS Reader codebase to understand its structure and functionality.
- Identify and document relevant system classes and components for the three major subsystems: Subscription and Content, Feed Organization, and User Management.
- Detect and justify design smells using tools like SonarQube and Designite Java, supported by manual inspection.
- Measure software quality through code metrics such as Cyclomatic Complexity, Code Duplication, and OOP-specific metrics (e.g., Chidamber and Kemerer metrics).
- Refactor the system based on identified design smells and metrics, ensuring adherence to software engineering principles.
- Compare manual refactoring with AI-generated suggestions using Large Language Models (LLMs) like GPT-4 or Claude.
- Automate the refactoring pipeline by integrating LLMs to detect design smells, refactor code, and generate pull requests.

3. System Overview

Rudra's Subscription Service (RSS) Reader is a web-based RSS aggregator that allows users to subscribe to and organize RSS feeds. The major subsystems include:

1. Subscription and Content Subsystem - Handles feed subscriptions, imports/exports, and displays recent articles.
2. Feed Organization Subsystem - Allows users to manage, categorize, and search for articles.
3. User Management Subsystem - Manages user accounts and authentication.

4. Task 1: Reverse Engineering

4.1. Identification of Relevant Classes

4.1.1. Subscription and Content Subsystem

1. SubscriptionResource (REST Resource)

- Handles feed subscription management via REST API
- *Functionality*: Add/update/delete subscriptions, import/export feeds (OPML/Google Takeout), fetch recent articles
- *Behavior*: Validates inputs, coordinates with DAOs for persistence, triggers feed synchronization

2. FeedService (Service)

- Core service for feed synchronization and maintenance
- *Functionality*: Periodic feed updates, feed parsing, article deduplication, favicon updates
- *Behavior*: Uses Quartz scheduler, coordinates with FeedDao/ArticleDao, handles feed parsing errors

3. Feed (Model)

- Represents RSS feed entity
- *Attributes*: RSS URL, website URL, title, description, synchronization status
- *Behavior*: Persisted via FeedDao, contains article relationships

4. Article (Model)

- Represents individual article content
- *Attributes*: Title, content URL, publication date, enclosures
- *Behavior*: Managed by ArticleDao, linked to parent Feed

5. OpmlReader (Utility)

- Parses OPML subscription files
- *Functionality*: Imports/Exports feed lists in standard OPML format
- *Behavior*: XML parsing, outline hierarchy processing

6. FeedSubscriptionDao (DAO)

- Manages user-feed subscription relationships
- *Functionality*: Subscription CRUD operations, unread count tracking
- *Behavior*: Uses FeedSubscriptionCriteria for queries, updates denormalized counts

7. FeedSynchronizationDao (DAO)

- Tracks feed update history
- *Functionality*: Records synchronization attempts and outcomes
- *Behavior*: Maintains performance metrics for feed updates

4.1.2. Feed Organization Subsystem

1. CategoryResource (REST Resource)

- Manages folder organization via REST
 - *Functionality*: Create/update/delete categories, manage article visibility
 - *Behavior*: Enforces folder hierarchy, coordinates with CategoryDao
2. **CategoryDao (DAO)**
 - Handles folder persistence and hierarchy
 - *Functionality*: Parent-child category relationships, ordering
 - *Behavior*: Uses nested set pattern for tree operations
 3. **UserArticleDao (DAO)**
 - Manages user-specific article states
 - *Functionality*: Read/unread tracking, starring, bulk operations
 - *Behavior*: Uses complex UserArticleCriteria for filtered queries
 4. **SearchResource (REST Resource)**
 - Provides full-text search capabilities
 - *Functionality*: Lucene-based article search
 - *Behavior*: Integrates with IndexingService, handles pagination
 5. **ArticleAssembler (DTO Mapper)**
 - Transforms domain models to API responses
 - *Functionality*: JSON serialization, field filtering
 - *Behavior*: Used by REST resources to format outputs
 6. **StarredResource (REST Resource)**
 - Manages starred articles
 - *Functionality*: Bulk starring/unstarring operations
 - *Behavior*: Coordinates with UserArticleDao for state changes
 7. **IndexingService (Service)**
 - Maintains search index
 - *Functionality*: Lucene index management, query processing
 - *Behavior*: Async index rebuilding, handles search pagination

4.1.3. User Management Subsystem

1. **UserResource (REST Resource)**
 - Exposes user management API
 - *Functionality*: Registration, login, profile updates
 - *Behavior*: Uses BCrypt password hashing, JWT token issuance
2. **UserDao (DAO)**
 - Manages user persistence
 - *Functionality*: CRUD operations, password hashing
 - *Behavior*: Enforces unique usernames, soft deletes
3. **AuthenticationTokenDao (DAO)**
 - Manages session tokens
 - *Functionality*: Token generation/validation, session cleanup

- *Behavior*: Supports both cookie and header-based auth
4. **SecurityFilter (Security)**
 - Authentication/Authorization filter
 - *Functionality*: Request validation, role checking
 - *Behavior*: Integrates with `UserPrincipal`/`IPrincipal`
 5. **UserPrincipal (Security)**
 - Represents authenticated user context
 - *Functionality*: Role-based access control
 - *Behavior*: Carries user permissions (`BaseFunction` enum)
 6. **PasswordChangedEvent (Event)**
 - Domain event for credential changes
 - *Functionality*: Triggers security updates
 - *Behavior*: Published through `AppContext` event bus
 7. **AppContext (Singleton)**
 - Manages application-wide services
 - *Functionality*: DI container, event bus coordination
 - *Behavior*: Initializes DAOs/Services, handles async operations

4.2. UML Class Diagram

4.2.1. Subscription and Content Subsystem

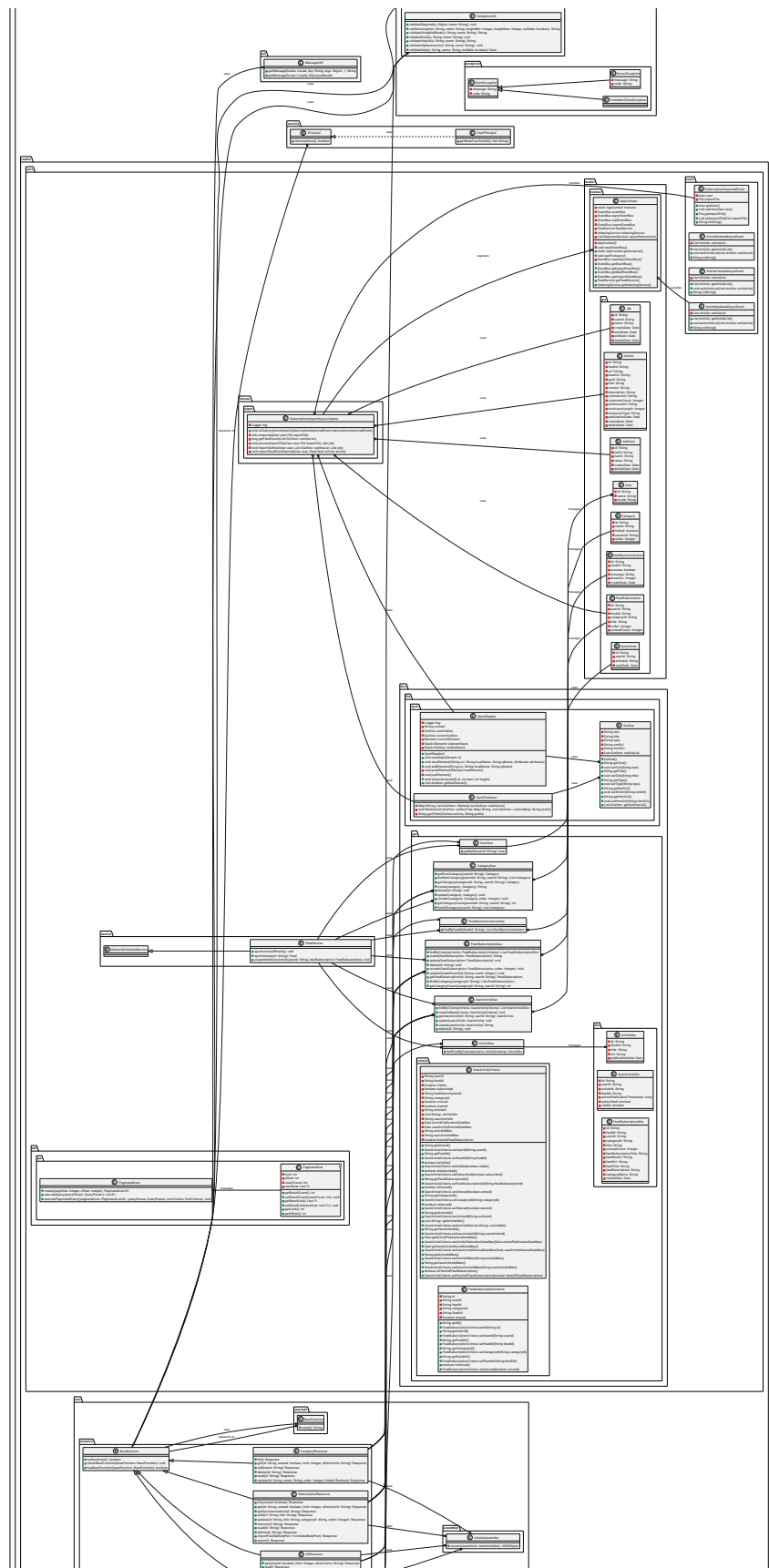


Figure 1: **Class Diagram of Subscription and Content Subsystem**

As per Figure 1 we can observe that ...

4.2.2. Feed Organization Subsystem

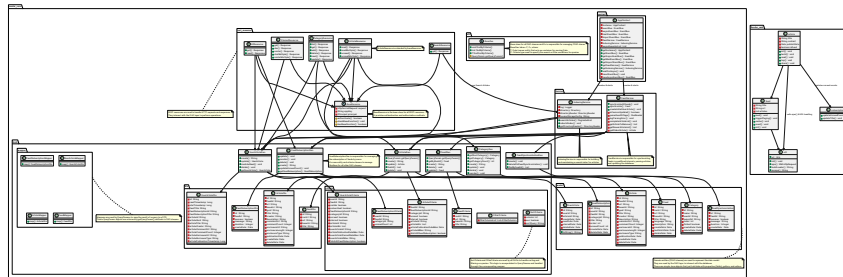


Figure 2: **Class Diagram of Feed Organization Subsystem**

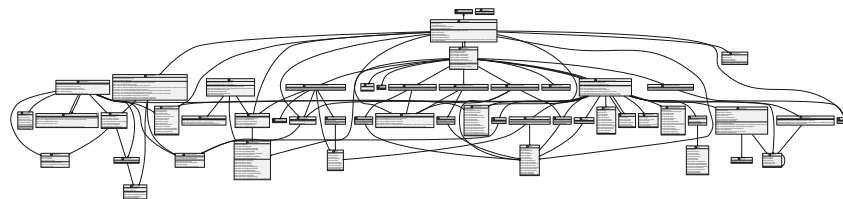


Figure 3: **Class Diagram of Indexing Service**

As per Figure 2 & Figure 3 we can observe that ...

4.2.3. User Management Subsystem

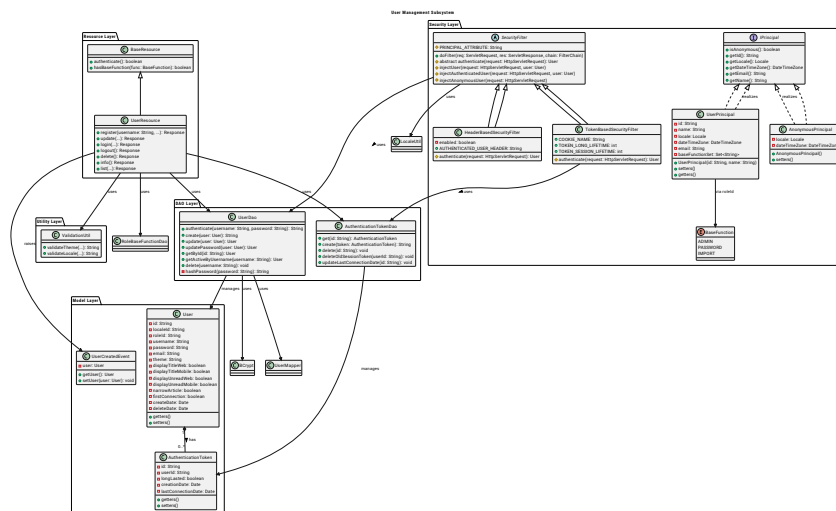


Figure 4: **Class Diagram of User Management Subsystem**

As per Figure 4 we can observe that ...

4.3. Observation and Comments

STRENGTHS

- **Modular Design:**
 - The system is well-organized into packages and sub-packages, making it easier to manage and maintain. Each package has a clear responsibility,

such as `reader-core` for core functionalities and `reader-web` for web-related functionalities.

- **Separation of Concerns:**
 - The system separates concerns like data access (DAO), business logic (services), and presentation (web resources). For example, `ArticleDao` handles database operations, while `ArticleResource` handles web requests.
- **Event-Driven Architecture:**
 - The system uses events (e.g., `ArticleCreatedAsyncEvent`, `FaviconUpdateRequestedEvent`) to handle asynchronous tasks, which improves scalability and decouples components.
- **Database and ORM Integration:**
 - The system uses JPA (Java Persistence API) for database interactions, which simplifies database operations and ensures consistency. The `BaseDao` class provides a common interface for CRUD operations.
- **Localization and Internationalization:**
 - The system supports multiple locales (e.g., `LocaleUtil`), making it adaptable to different languages and regions.

WEAKNESSES

- **Complexity:**
 - The system is quite complex, with many layers (e.g., DAO, services, resources) and dependencies. This complexity can make it difficult for new developers to understand and contribute to the codebase.
- **Tight Coupling in Some Areas:**
 - While the system generally follows good design principles, there are areas where components are tightly coupled. For example, some classes directly create instances of other classes, which can make testing and maintenance harder.
- **Potential Performance Bottlenecks:**
 - The system relies heavily on database operations and synchronous processing in some areas (e.g., `FeedService`), which could lead to performance bottlenecks, especially under heavy load.
- **Lack of Documentation:**
 - The UML diagram does not provide detailed documentation for each class or method. This lack of documentation can make it difficult to understand the purpose and usage of certain components.
- **Limited Scalability in Some Areas:**
 - Some components, like `FeedService`, may not scale well if the number of feeds or users grows significantly. The system might need to be optimized for large-scale deployments.

- **Hardcoded Values:**
 - Some classes contain hardcoded values, which reduces flexibility and makes it harder to configure the system for different environments.
- **Testing Challenges:**
 - The system's complexity and tight coupling in some areas could make unit testing and integration testing challenging. Mocking dependencies and ensuring test coverage might require significant effort.

SUMMARY

The system has complexity, tight coupling in some areas, and potential performance bottlenecks could pose challenges. Improving documentation, reducing hardcoded values, and optimizing for scalability would enhance the system further.

5. Task 2: Design Smells and Code Metrics

5.1. Design Smell Analysis

Design Smell	Description	Justification
Deficient Encapsulation	This smell occurs when the declared accessibility of one or more members of an abstraction is more permissive than actually required.	The class fields or methods are not properly encapsulated, leading to potential misuse or unintended access from outside the class. https://github.com/SE-course-serc/project-1-team-17/issues/4
Primitive Obsession / Broken Modularization (as per Designite)	This smell occurs when primitive data types are used where an abstraction encapsulation the primitives could serve better	MIME types are represented as bare string constants. https://github.com/SE-course-serc/project-1-team-17/issues/5
Imperative Abstraction	This smell arises when an operation is turned into a class	The tool detected this design smell in 3 classes and we merged them into a single class https://github.com/SE-course-serc/project-1-team-17/issues/6
God Class	This smell occurs when a class is large containing too many variables and methods	The ApplicationContext was a God Class that was refactored by implementing EventBusManager and AsyncTaskManager https://github.com/SE-course-serc/project-1-team-17/issues/7
Cyclic Dependency	This smell arises when two or more abstractions depend on each other directly or indirectly	https://github.com/SE-course-serc/project-1-team-17/issues/8

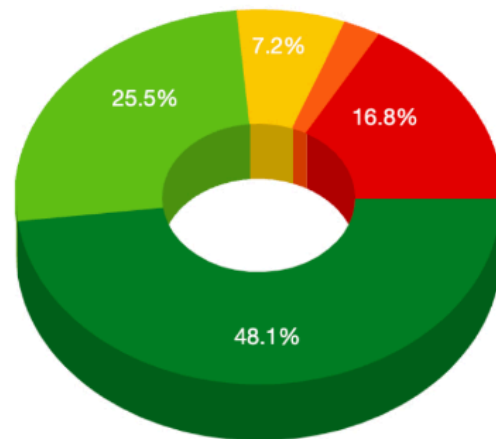
Table 1: **Design Smells Identified in the RSS Reader**

5.2. Code Metrics

Tools Used

- Sonarqube
- CodeMR
- Designite

Extracted Metrics



C3

- Very High
- High
- Medium-high
- Low-medium
- Low

Figure 5: **C3 before refactoring**

Distribution of Quality Attributes

Complexity, Coupling, Cohesion, and Size



Figure 6: Metrics before refactoring

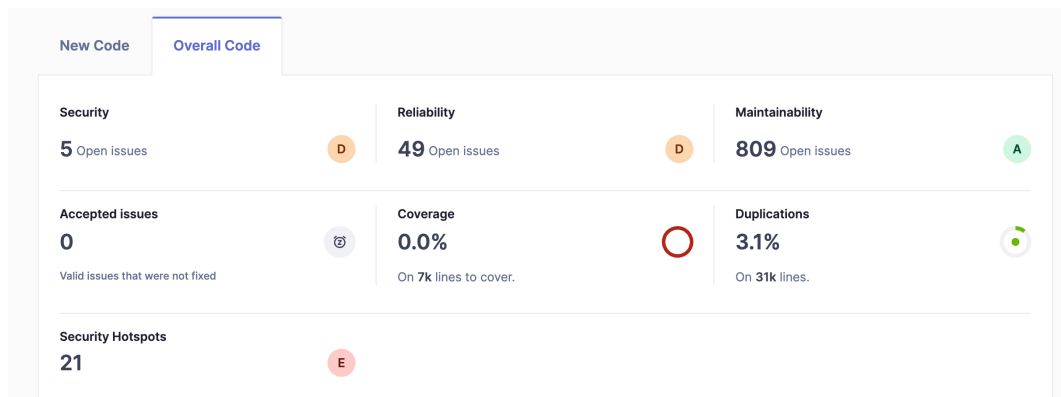


Figure 7: SonarQube Stats before refactoring

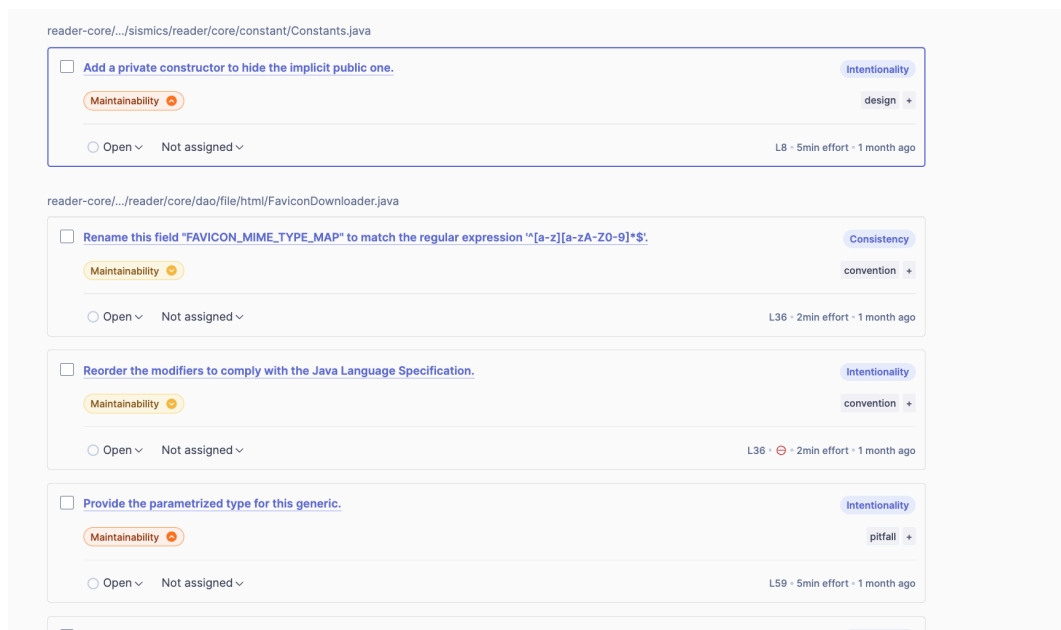


Figure 8: SonarQube Issues before refactoring

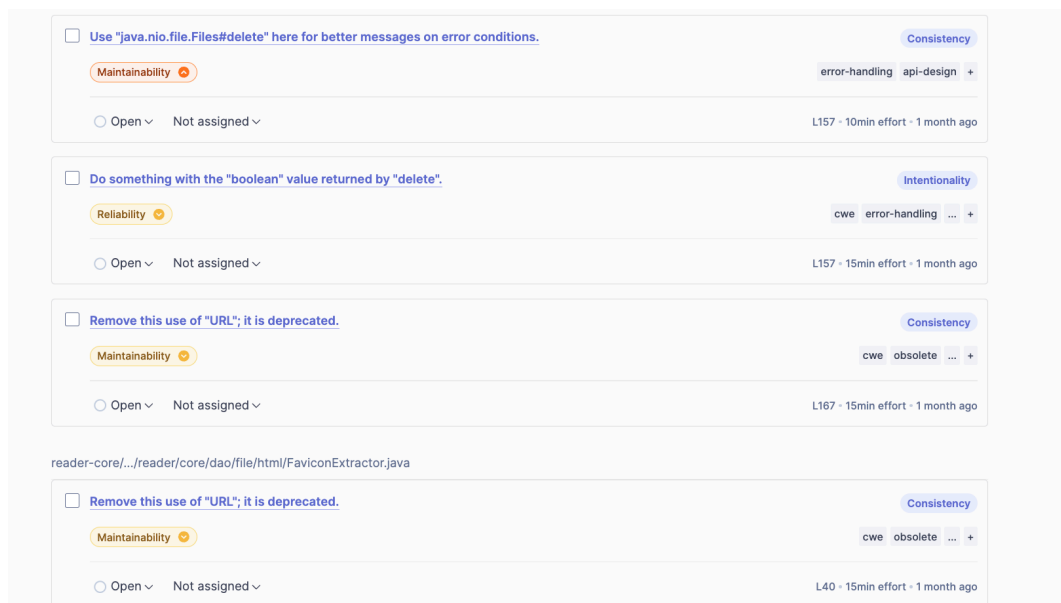


Figure 9: SonarQube Issues before refactoring

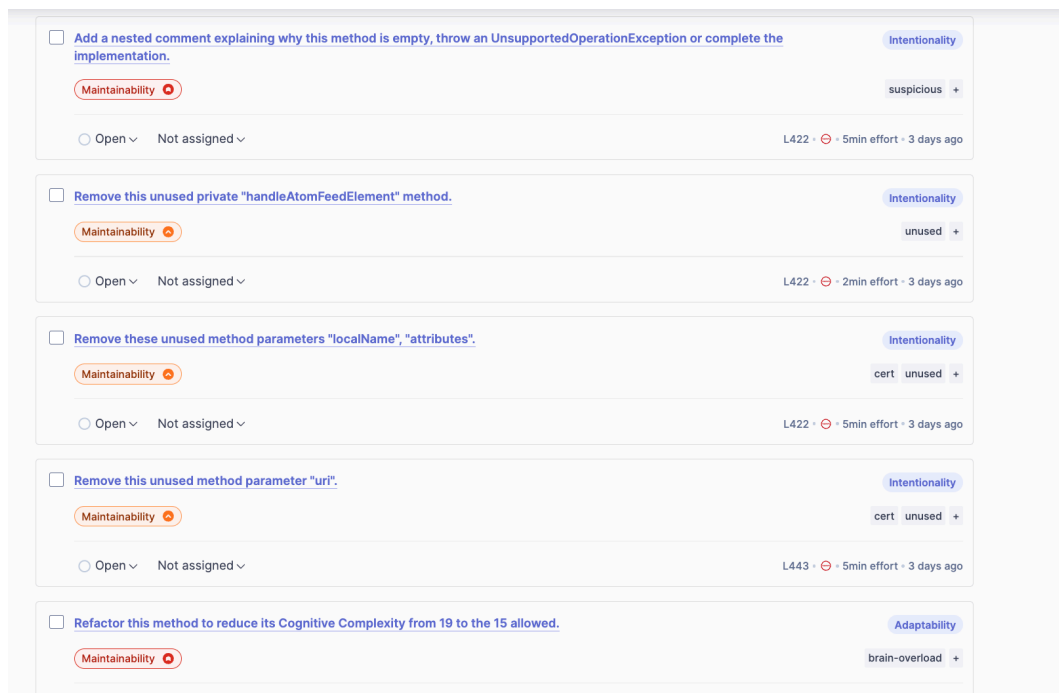


Figure 10: SonarQube Issues before refactoring

5.3. Implications Discussions

For the identified metrics, the following are their implications in terms of software quality, maintainability, and potential performance issues:

- **Coupling**

Coupling refers to the degree of interdependence between classes, characterized by various relationships such as attributes, method calls, and inheritance. Tightly coupled systems often lead to a ripple effect of changes, requiring more effort to maintain and making classes harder to reuse due to their dependencies.

- **Lack of Cohesion**

Lack of cohesion measures how related the methods of a class are to one another. High cohesion is desirable as it enhances software traits like robustness and understandability, while low cohesion indicates that a class may be handling multiple responsibilities, making it difficult to maintain and test.

- **Complexity**

Complexity reflects how difficult it is to understand the interactions among various entities within the software. Increased complexity raises the risk of unintended interactions, which can lead to defects when changes are made, thereby complicating maintenance and development.

- **Size**

Size is a traditional metric for software measurement, typically assessed by counting lines of code or methods. A high size count may suggest that a class or method is overloaded with responsibilities, indicating a need for refactoring to improve maintainability.

- **Weighted Method Count**

The Weighted Method Count (WMC) quantifies the complexity of a class by summing the complexities of its methods. A high WMC can indicate that a class is domain-specific and less reusable, as well as more prone to changes and defects, especially if it inherits many methods from a base class.

- **Lack of Cohesion of Methods (LCOM3)**

LCOM3 measures the interrelatedness of methods within a class, with low cohesion suggesting that the class is handling multiple responsibilities. High LCOM3 values indicate a need for refactoring, as they imply that the class should be split into smaller, more focused subclasses to enhance understandability and maintainability.

- **C3**

C3 is a composite metric that captures the maximum values of coupling, cohesion, and complexity within a class. This metric provides a holistic view of a class's design quality, helping to identify potential areas for improvement in software architecture.

6. Task 3: Refactoring

6.1. Addressing Design Smells

We discovered many design smells using Designite, and we refactored few of them; however, some were false positives.

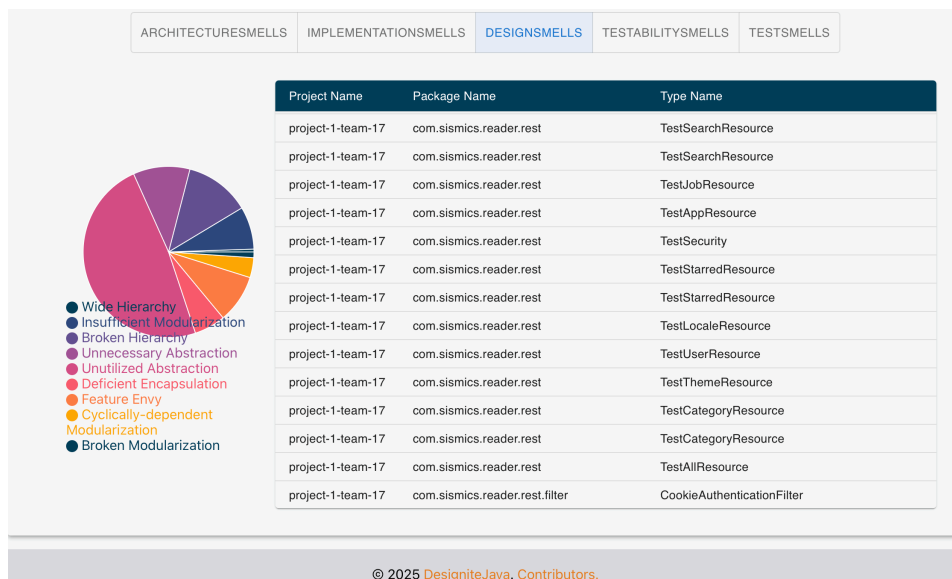


Figure 11: Design smells given by Designite

Following were the design smells we addressed:

<https://github.com/SE-course-serc/project-1-team-17/issues?q=is%3Aissue>

6.2. Code Metrics Post Refactoring

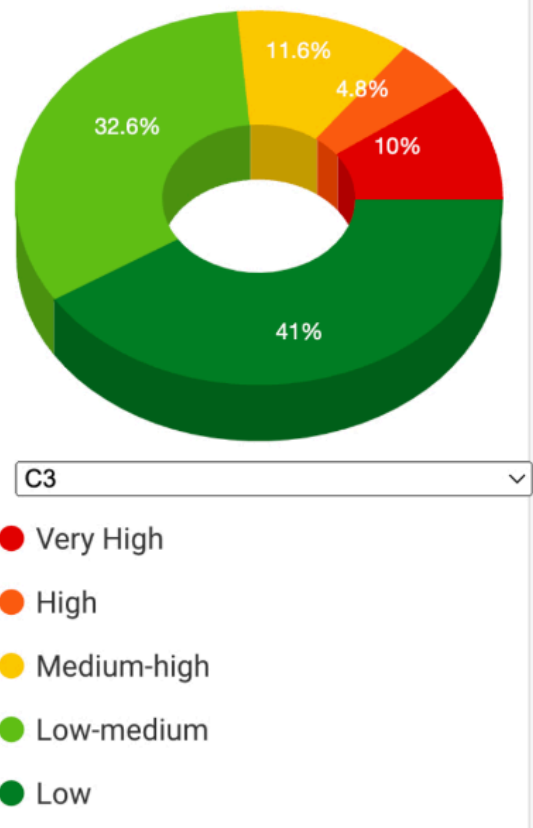


Figure 12: C3 after refactoring

Distribution of Quality Attributes

Complexity, Coupling, Cohesion, and Size

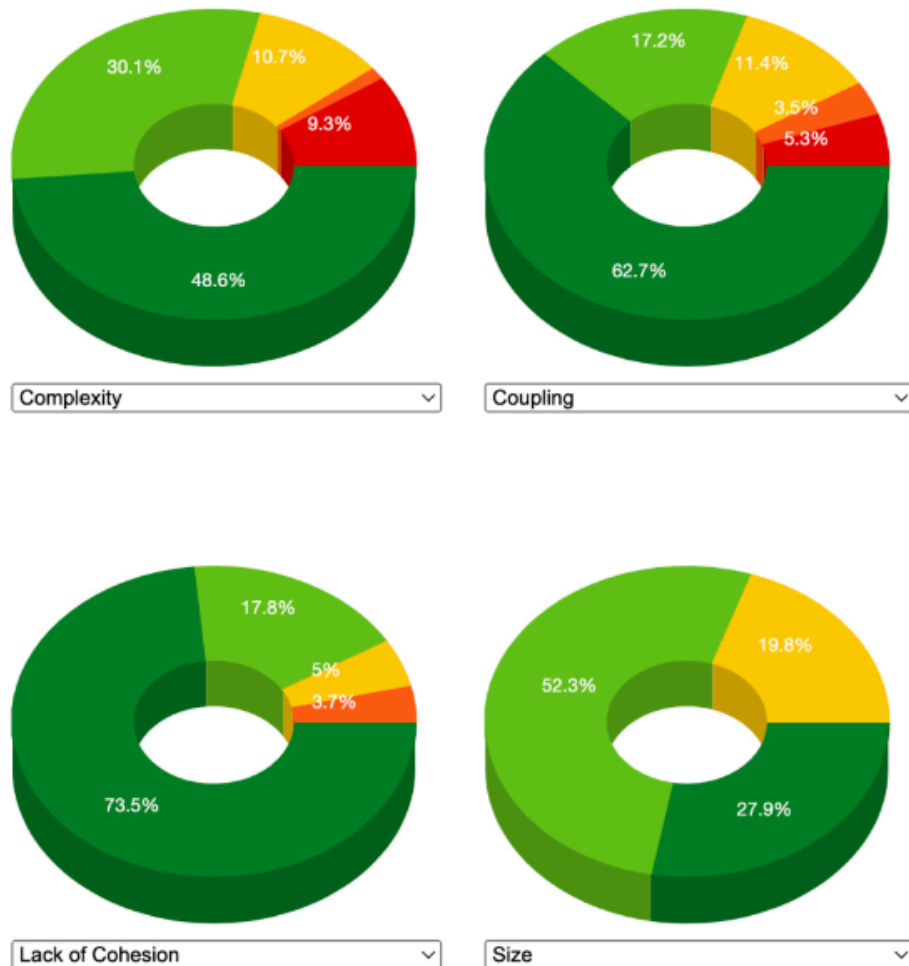


Figure 13: **Metrics after refactoring**

6.3. Analysis of Code Metrics after Refactoring

After refactoring the code, we observed noticeable improvements in most of the code metrics. Many of the issues related to design smells were addressed, and adherence to clean code practices and SOLID principles resulted in enhanced maintainability and readability. This overall trend aligns with our expectations, as better design usually reduces complexity and coupling while increasing cohesion.

However, not all metrics showed improvement; some tradeoffs became evident during the refactoring process. For instance, while complexity and cohesion improved overall, certain changes led to a slight increase in size or coupling in specific areas. This outcome is not uncommon since optimizing one aspect of the

code can sometimes impact another. In summary, although most metrics improved it is rare to see an all over enhancement in every metric due to inevitable tradeoffs in software architecture.

6.4. Leveraging LLMs for Refactoring

LLM based Refactoring Analysis

We utilized DeepSeek to refactor one of the design smells we identified and compared the results with our manual refactoring. We addressed the design smell manually in this issue: <https://github.com/SE-course-serc/project-1-team-17/issues/7>, and below are the suggestions provided by DeepSeek:

Suggested Refactoring Approach:

1. Encapsulate Event Bus Management:

- Create `EventManager` to handle event bus creation and listener registration, but encapsulate event posting behind methods.
- Use interfaces for event buses to enable easier mocking in tests.

2. Improve `AsyncTaskManager`:

- Centralize thread pool configuration and enforce lifecycle management (e.g., `PreDestroy` hooks for shutdown).
- Use distinct executors for different event types (mail, imports) with appropriate configurations.

3. Simplify `AppContext`:

- Remove all event bus/async logic; delegate entirely to `EventManager` and `AsyncTaskManager`.
- Use constructor injection for dependencies instead of a static `initialize` method.

4. Code Modernization:

- Use Java's `Executors` utilities for thread pools.
- Replace legacy `Lists.newArrayList()` with modern `new ArrayList<>()`.

— DeepSeek

Example Refactored Code Snippets:

```

1 // EventBusManager.java (simplified)
2 public class EventBusManager {
3     private final EventBus asyncEventBus;
4     private final EventBus mailEventBus;
5
6     public EventBusManager(AsyncTaskManager taskManager) {
7         this.asyncEventBus = new
8 AsyncEventBus(taskManager.createExecutor("async"));
9         this.mailEventBus = new
10 AsyncEventBus(taskManager.createExecutor("mail"));
11     registerListeners();
12 }
13
14 private void registerListeners() {
15     asyncEventBus.register(new ArticleCreatedAsyncListener());
16     // ... other listeners
17 }
18
19 // Encapsulate posting events:
20 public void postAsyncEvent(Object event) {
21     asyncEventBus.post(event);
22 }
23 }

```

```

1 // AsyncTaskManager.java
2 public class AsyncTaskManager {
3     private final Map<String, ExecutorService> executors = new
4 ConcurrentHashMap<>();
5
6     public ExecutorService createExecutor(String type) {
7         ExecutorService executor =
8 Executors.newFixedThreadPool(2); // Example config
9         executors.put(type, executor);
10        return executor;
11    }
12
13    @PreDestroy
14    public void shutdown() {
15        executors.values().forEach(ExecutorService::shutdown);
16    }
17 }

```

```

1 // AppContext.java (minimalist)
2 public class AppContext {
3     private final FeedService feedService;
4     private final EventBusManager eventBusManager;
5
6     public AppContext(FeedService feedService, EventBusManager
    eventBusManager) {
7         this.feedService = feedService;
8         this.eventBusManager = eventBusManager;
9     }
10
11     // No static initialization logic; use DI.
12 }

```

– DeepSeek

Comparison with Our Version

Aspect	Our Version	Suggested Version
Clarity	Clear separation via EventBusManager and AsyncTaskManager	Enhanced by encapsulating event posting (e.g., postAsyncEvent() vs. exposing buses).
Conciseness	Direct but exposes internal event buses (e.g., getAsyncEventBus())	More concise API surface; clients don't interact directly with Guava classes.
Best Practices	Follows SRP but uses static initialization.	Uses constructor injection and lifecycle hooks (e.g., PreDestroy for shutdown).
Testability	Relies on AppContext.getInstance(), which is harder to mock	Constructor injection allows easier mocking in unit tests.
Resource Management	Manually calls waitForAsyncCompletion()	Automates shutdown via PreDestroy and consolidates executors by type.

Table 2: **Comparison b/w our vs LLM refactoring**

6.5. Automation of Refactoring Pipeline

TODO: Automated Pipeline to be done @Nitheesh

7. Acknowledgements

7.1. Contributions

Following are the contributions made by the team:

- UML Design: Sanket, Priyank, Aditya, Nitheesh, Yash
- Identifying Smells: Sanket, Priyank, Aditya
- Code Metrics: Sanket, Nitheesh
- Manual Refactoring: Sanket, Aditya
- LLM Refactoring: Priyank, Aditya
- Automating Refactoring: Nitheesh, Sanket
- Documentation: Sanket, Priyank, Nitheesh

Appendices

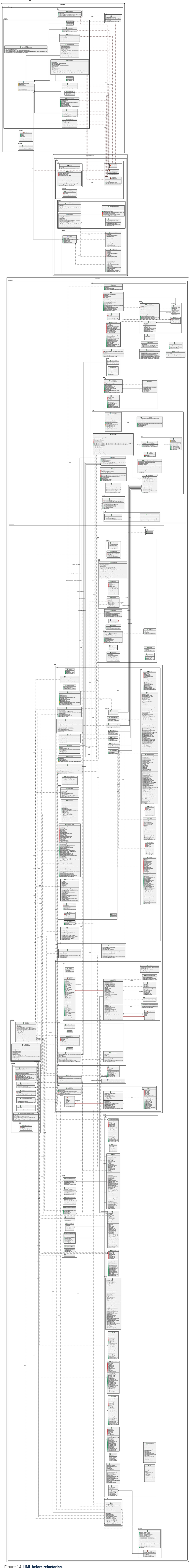


Figure 14: UML before refactoring.